SOE104 Oregon Trail — Engine Rules Specification

Every number the engine uses, exactly as it runs. Nothing is hidden: strategy beats dice.

1. Journey frame

PARAMETER	VALUE
Total distance	2,000 miles
Turn limit	120 turns
Shared seeds	3 (identical for every team; set per assignment, default {1, 2, 3})
Difficulty multiplier	1.0 (fixed; all severity math scales by it)

End conditions. The journey ends when the party (a) reaches 2,000 miles, (b) exhausts 120 turns, or (c) has zero living members (immediate loss). Zero oxen does not end the run — the wagon is stranded (no travel) while turns and hunger keep running.

Turn order (fixed, 7 phases):

1. **Strategy** — `PartyStrategy.decide(PartyState)` → `Directive`
2. **Travel** — pace miles (requires ≥ 1 ox); grueling fatigue & ox risk
3. **Feeding** — members eat rations, then oxen eat; starvation accounting
4. **Event** — one random event may strike; `Resolver.resolve(Event)` answers it
5. **Member acts** — every living member performs one `Action`
6. **Bookkeeping** — resource end-of-turn hooks; trace snapshot
7. **Survival check** — all-dead ends the run

---

2. The General Store

A one-time shopping trip before turn 1. Every team gets the same flat wallet — party size does NOT change it.

RULE	VALUE
Wallet	**$2,000** (exactly the price of the classic wagonload + $300)
Goods	food $2, ammo $3, spare parts $30, oxen $80
Buying	`PartyFactory.buySupplies()` → `List<Purchase>`, e.g. `new Purchase(ResourceType.OXEN, 6)`
Processing	purchases applied **in list order**; each quantity clipped to what the wallet still affords; a good you cannot afford one unit of is skipped
Change	unspent wallet remains as starting **money**
Default	`null` (or empty list) → engine buys the **classic basket**: 500 food, 80 ammo, 5 oxen, 2 parts — $300 change.
Restrictions	buying only — no selling, no haggling at the store; food purchases also clip to the wagon's food capacity (§2.1)

2.1 Wagon capacity

The wagon carries at most **500 food**. Every source — store, HUNT, hunt opportunities, market purchases — is clipped to capacity; the surplus spoils (the ammo is still spent). Hoarding is impossible; just-in-time procurement is the skill.

The store callback runs inside the sandbox under the same step budget as `act()`.

---

3. Party and roles

- `createMembers()` must return **3–5 members**. Anything else fails the run with an explicit message (functional gate).
- Members are constructed with `super(name)` (full health) or `super(name, hp)` (0–100; out of range throws).
- `act()` may return `null` — the member idles that turn (recorded in replay).

3.1 Role classes and perks

Five role classes ship in `oregon-api`, each an empty subclass of `Member` with constructors `Role(String name)` and `Role(String name, int hp)`. **Three perks change the member's own actions (Hunter, Doctor, Scout); two are auras the engine applies while the member lives (Mechanic, Driver) — no code needed, just keep them alive.**

ROLE	PERK (APPLIES TO THE MEMBER'S OWN ACTION)
`Hunter`	own HUNT yields **10** food per ammo spent (base 7)
`Doctor`	own HEAL base **+15** (base +10) and treats the party's most wounded member instead of only themself
`Mechanic`	aura: while any Mechanic lives, BREAKDOWN **SAFE** repairs consume no spare part
`Scout`	own SCOUT finds **+6 miles** (base +3)
`Driver`	aura: while any Driver lives, GRUELING costs **2 HP** per member (not 4) and ox-death chance is **7%** (not 15%)

- Duplicates are fine: each self-perk copy boosts its own actions (two Hunters each hunt at 10; they do not boost each other). Auras do not stack — one living Mechanic or Driver is enough. Subclassing `Member` directly works for every action but grants no perk.

Members read shared supplies through `PartyState.getResources()` (for strategies) and `Situation` (for `act()`).

---

4. Member actions

`Action.of(Type)` / `new Action(Type)` — actions are binary: a member chooses *what* to do, never how hard. Every yield, cost and perk below is a fixed rule. (`new Action(null)` throws, which fails the run — program defensively.)

ACTION	EFFECT
`HUNT`	costs **1 ammo**; yields `7` food (Hunter: `10`), clipped to wagon capacity (§2.1)
`HEAL`	self `+10` HP; a Doctor's HEAL instead treats the **most wounded living member** (themself if sickest) at `+15`; **no effect on a starving turn**
`REST`	self `+3` HP; same starvation rule
`SCOUT`	`+3` miles this turn (Scout: `+6`) — no HP cost
`MOVE`	`+5` miles this turn, self `−1 HP` — for everyone, Drivers included
`DEFEND`	if a threatening event hit this turn: every living member `+3` HP. **Triggers at most once per turn** no matter how many members DEFEND.

Member actions resolve in roster order after the event; each member sees the event in `Situation.getCurrentEvent()` and may react the same turn.

---

5. Party directives

`new Directive(Pace, Ration)`.

DIMENSION	OPTIONS
Pace	SLOW 12 / NORMAL 20 / GRUELING 28 miles (at ≥ 4 oxen)
Rations	BARE 1 (slow attrition: −1 HP per member per turn) / MEAGER 2 / FULL 5 (+2 HP per member when fully fed)

- **Speed:** `miles = pace × min(1, oxen / 4)`; 0 oxen → no travel.
- **Grueling:** every living member −4 HP that turn, plus a 15% chance one ox dies of exhaustion. **While a Driver lives**: −2 HP and 7% instead.
- **FULL rations, fully fed:** every living member +2 HP that turn.
- **Engine default** when `decide()` returns null or throws: `NORMAL` pace + `FULL` rations — a food-hungry default; an engaged strategy does better.

- **Oxen feed** after members: 2 food per ox per turn (oxen are expensive to keep — oxen count is a real tradeoff). If food runs out mid-turn, oxen simply go unfed.

---

6. Feeding and starvation

1. Living members eat `ration × living` food total.
2. Oxen eat `2 × oxen` food.
3. If fewer members were fully fed than are alive (integer semantics — partial feeding counts as hunger), **every member** takes escalating damage and `HEAL` / `REST` are void that turn:

CONSECUTIVE HUNGRY TURN	1	2	3	4	5+
Damage to every member	5	8	12	17	23 (capped)

4. A fully-fed turn resets the counter.

---

7. Random events

Schedule. Each turn: `chance = 15% + 28% × progress` (progress = distance ÷ 2000; ~15% early → ~43% late — events are frequent, and the Resolver is a first-class decision surface). On a hit, severity = `1..5 (uniform) + round(6 × progress)`, clamped to [1, 10]. Write `s` for severity. All event HP damage hits **every living member**.

Kind pools by phase (current engine, verbatim):

PROGRESS	POOL
0–25%	HUNT_OPP ×2, BAD_TRAIL, TRADER, STORM
25–60%	STORM, BREAKDOWN, THEFT, TRADER, HUNT_OPP, BAD_TRAIL
60–90%	RIVER, ILLNESS, ATTACK, BAD_TRAIL, STORM, BREAKDOWN, TRADER
90–100%	RIVER, ATTACK, ILLNESS, STORM, THEFT, BREAKDOWN, TRADER

Global fairness rule: a SAFE option whose money price you cannot afford is **downgraded to AVOID** automatically (applies to RIVER ferry, ATTACK flee). No more free ferries for broke parties.

7.1 The event catalogue

`s` = severity (1–10). All HP damage hits every living member. ### 🌊 RIVER — a river blocks the way (severity = depth)

- **SAFE** — pay the ferryman: **−$25** (must have it; if you cannot afford it, this choice is treated as AVOID)
- **RISKY** — ford the river: `s/10` chance the party takes **−10 HP** and **−20 food**; then a **20%** chance an ox drowns
- **AVOID** — wait on the bank: **−5 miles**

🌨️ STORM — bad weather rolls in

- **SAFE** — shelter and ride it out: party **−2 HP**
- **RISKY** — push through: party **−s HP**; **15%** chance an ox dies

- **AVOID** — go around: **−10 miles**

🛠️ BREAKDOWN — something on the wagon cracks

- **SAFE** — repair: consumes **1 spare part** — but with a living **Mechanic** the repair costs no part; with no parts left, same as AVOID (**−15 miles**)
- **RISKY** — jury-rig: `s/10` chance the party takes **−5 HP** and **−1 part**
- **AVOID** — abandon weight: **−15 miles**

🤒 ILLNESS — sickness sweeps the party

- **SAFE** — rest it off: **−3 × living members food**, **−10 miles**
- **RISKY** — push through: `s/10` chance the party takes **−(s+5) HP**, otherwise shakes it off entirely
- **AVOID** — endure it: party **−s HP**, sure thing

🐎 ATTACK — bandits or wildlife

- **SAFE** — flee: **−$30** (must have it; otherwise treated as AVOID) and **−10 food**
- **RISKY** — fight: party **−s HP**
- **AVOID** — hide: **−15 miles**

TRADER — a market opens

- **SAFE** — trade at posted prices: `buy(good, qty)` (clipped to money, **≤50 units/good/visit**, and food to wagon capacity)
- **RISKY** — haggle: success chance `0.5 + (s−5)×0.04` (clamped 0.2–0.9); success → buy ×0.7 / sell ×1.3; failure → no deal
- **AVOID** — walk away

🦌 HUNT_OPP — abundant game spotted

- **SAFE** — careful hunt: **−2 ammo** → **+20 food** (needs the ammo)
- **RISKY** — big hunt: **−4 ammo** → **+50 food**, `s/10` chance the party takes **−5 HP** (needs the ammo)
- **AVOID** — pass it up

⛰️ BAD_TRAIL — the trail forks

- **SAFE** — long detour: **−10 miles** and **−5 food** (the long road eats supplies)
- **RISKY** — shortcut gamble: `s/10` chance the party takes **−s HP** and **15%** an ox dies; otherwise **+5 miles**
- **AVOID** — turn back: **−20 miles**

🥷 THEFT — a thief circles the camp

- **SAFE** — post guards: **−10 food**
- **RISKY** — chase: 50/50 — party **−5 HP**, or **−20 food** and **−$20**
- **AVOID** — let them take the cash box: **−$15**

All event HP damage hits **every living member**. Food losses clip to stock; money price tags follow the affordability rule above.

---

8. Trail markets (TRADER)

One transaction per visit:

- **Prices:** posted per good = `base price × band`, where band = ×0.7 (severity 1–3, bargain), ×1.0 (4–6, fair), ×1.4 (7–10, steep); rounded, min $1. Sell price = 70% of buy price (rounded, min $1).
- **Buy:** quantity clipped to what your money affords, to **50 units per good per visit**, and (for food) to wagon capacity; cannot afford one unit → no deal.
- **Sell:** quantity clipped to stock; no stock → no deal.
- **Haggle** (RISKY factories): success chance = `clamp(0.5 + (s − 5) × 0.04, 0.2, 0.9)`; success → buy ×0.7 / sell ×1.3; failure → the deal is off entirely.

---

9. Scoring

```
FINAL = 0.20·OOP + 0.50·Game + 0.15·Naming + 0.15·Efficiency     (3-seed average)
Game  = 60%·(distance/2000) + 25%·(survivors/initial) + 15%·min(1,
totalUnits/1000)·(distance/2000)
```

Supplies score only in proportion to distance covered — dying rich is not an achievement.

- **Functional gate:** compile failure, crash, or timeout → Game = 0 and FINAL capped at 40. Banned API (§10) → 0.
- **Efficiency bands** (absolute, generous): ≤8,000 avg ops/seed → 100, ≤25,000 → 80, ≤60,000 → 60, else ≤40. Legitimate decision logic measures 100–6,000 ops (a 560-line reference implementation with shared-context composition measures ~5,700); the dimension exists to catch pathological computation (runaway loops measure in the millions), not richness.
- **OOP subscale** (bytecode static analysis, checklist published to students): encapsulation 25, inheritance 25, polymorphism 25, interfaces 15, cohesion 10.
- **Naming:** PascalCase classes, camelCase members, no single-letter or vague names; full checklist published.

---

10. Sandbox and determinism

- JDK 21; student compile classpath contains only `oregon-api`.
- Submission: `.java` files or one `.zip`, files at archive root, **no packages**, UTF-8, one public class per file, **1–20 `.java` files**, ≤ 20 MB . Only `.java` entries are extracted from archives — anything else in the zip is ignored, so bytecode cannot be smuggled in.
- **Banned APIs** (bytecode scan; any hit → submission rejected, FINAL 0):

CATEGORY	BAN
Reflection	`java.lang.reflect.*`, `Class.forName`, `java.lang.invoke.*`
Subprocess	`Runtime`, `ProcessBuilder`
Threads	`Thread` construction/start, `java.util.concurrent.*`
Classloading	custom `ClassLoader`
Exit	`System.exit`
Randomness	`java.util.Random` (instrumented `Math.random()` is deterministic and allowed)
Engine-owned state	any direct modification of member health, supplies or the roster — these change only through the engine applying the submission's actions and choices
Files	`java.io.*`, `java.nio.*` — sole exception: console printing (`System.out` / `System.err`)
Network	`java.net.*`
Environment	`System.getenv`, `System.getProperty`

- **Budgets:** 100,000 counted operations per student callback (over-budget call is skipped that turn, replay notes it); 10 s wall clock per seed in a child JVM (`-Xmx256m`), then force-killed.
- **Determinism:** `Math.random()` and time APIs are rewritten by instrumentation to seeded sources; all teams run identical seeds; identical submissions always score identically.
- Child JVM hardening: the child runs in an empty scratch working directory with `-Djava.io.tmpdir` scoped to it, and a randomized result filename.
